

Verteilte Systeme

12. Juli 2026

1 Aufgabe 1: Eine kleine verteilte Anwendung (das "Bankhaus")

Bei der Aufgabe wurde zunächst der Simulator gestartet. Für den Testlauf wurden vier Bankkonten gestartet. Sobald alle Prozesse gestartet sind, kann der Koordinator damit beginnen Überweisungen zu tätigen. Einmal pro Sekunde bestimmt der Koordinator (Konto 0) ein zufälliges Konto. Sobald dieses Konto die Nachricht erhalten hat, dass es jetzt dran ist mit der Überweisung von Geld, bestimmt es einen zufälligen Partner sowie eine zufällige Menge von Geld. Erhält ein Partner eine Summe, so erhöht er seine eigene Balance. Versendet ein Partner Geld, reduziert er den entsprechenden Betrag von seinem Konto. Der genaue Ablauf ist der Abbildung zu entnehmen.

1.1 Implementierung

Der Ablauf startet in main: Dort wird das Simulator-Singleton geholt, es werden vier ProcessNode-Objekte mit je 1000 € Startguthaben erzeugt, und `simulate(3)` lässt die Simulation drei Sekunden laufen, indem bei jedem Knoten `engage()` aufgerufen wird. Nur der Koordinator (der Knoten mit `id == 0`) startet zusätzlich einen Thread, der alle 100 ms ein Token an einen zufällig gewählten Knoten schickt. Jeder Knoten wartet in einer Schleife auf Nachrichten und verarbeitet sie je nach Typ per switch. Trifft ein Token ein und ist das Guthaben größer als null, wählt der Knoten einen zufälligen Betrag und einen zufälligen Empfänger (der nicht er selbst ist), zieht den Betrag von seinem Konto ab und sendet einen Transfer an diesen Empfänger. Der Empfänger erhält den Transfer, prüft, ob er wirklich gemeint ist, und schreibt sich den Betrag gut. Dieses Token- und Transfer-Spiel wiederholt sich fortlaufend, sodass ständig Geld zwischen den Knoten hin- und herwandert. Nach Ablauf der drei Sekunden beendet `shutdown()` die Simulation.

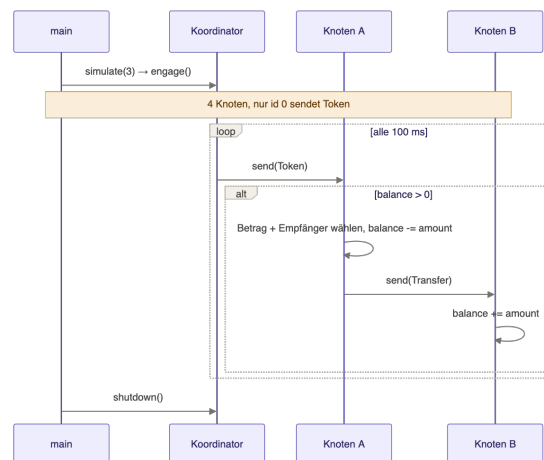


Abbildung 1: Sequenzdiagramm

2 Aufgabe 2: Implementierung des Schnappschuss-Algorithmus

Um den Snapshot-Algorithmus zu implementieren, sind die folgenden Records hinzugekommen:

- `SnapshotStart()`
- `SnapshotReport()`
- `ChannelReport()`

Die Implementierung zum Bankaccount bleibt im Großen und Ganzen identisch. Nach einer Sekunde wird allerdings die Snapshotanfrage vom Koordinator verschickt: zunächst an sich selbst und dann an alle anderen. Beim Durchführen des Snapshots ändert der Knoten seine Farbe von weiß zu schwarz und erstellt den Report, hält also seinen lokalen Kontostand fest.

Ein zentraler Punkt ist die Behandlung der Nachrichten-Einfärbung: Jede **Transfer**-Nachricht trägt die Farbe ihres Senders zum Sendezeitpunkt. Empfängt ein bereits schwarzer Prozess eine weiße Nachricht, so war dieses Geld zum Schnittzeitpunkt unterwegs und wird als Kanalzustand per **ChannelReport** an den Koordinator gemeldet. Empfängt umgekehrt ein noch weißer Prozess eine schwarze Nachricht, führt er zuerst seinen eigenen Snapshot durch, sodass der Betrag genau einmal (beim Sender) erfasst wird. Die Methode `checkDone()` gibt den Schnappschuss aus, sobald alle n lokalen Zustände gemeldet wurden und die Summe aus Kontoständen und Kanalgehalten dem bekannten Gesamtbetrag S entspricht. Da die FIFO-Zustellung abgeschaltet ist, kann eine schwarze Basisnachricht die Kontrollnachricht überholen; dieser Fall wird dadurch abgefangen, dass der Empfang einer schwarzen Nachricht selbst einen Snapshot auslöst. Somit hängt die korrekte Zuordnung „vor bzw. nach dem Schnitt“ ausschließlich an der Farbe der Nachricht und nicht an der Reihenfolge im Kanal. Auf diese Weise erfasst der Algorithmus einen konsistenten globalen Zustand, der die Invariante $\sum_i \text{balance}_i + \text{Kanalinhalt} = S$ erfüllt.

Der Algorithmus setzt die üblichen Annahmen des Verfahrens voraus: Es gehen keine Nachrichten verloren und kein Prozess stürzt ab, sodass jede abgeschickte Überweisung den Empfänger auch erreicht. Da wir die FIFO-Zustellung über `setMessageQueueSelectionDistributionFunction` abgeschaltet haben, kann eine schwarze Basisnachricht die **SnapshotStart**- Kontrollnachricht auf demselben Kanal überholen. Im klassischen Chandy-Lamport-Verfahren (Variante b) wäre dies problematisch, da dort die Kanalaufzeichnung genau durch das Eintreffen des zweiten Markers begrenzt wird und somit FIFO benötigt. Beim gewählten Einfärbeverfahren (Variante a) trägt hingegen jede Nachricht ihre Farbe explizit mit, sodass die Zuordnung „vor bzw. nach dem Schnitt“ allein an der Farbe und nicht an der Kanalreihenfolge hängt. Das Überholen wird zusätzlich dadurch korrekt behandelt, dass eine schwarze Nachricht bei einem noch weißen Empfänger dessen Snapshot auslöst, bevor sie verbucht wird. Damit bleibt die Kanalaufzeichnung auch ohne FIFO korrekt.

3 Aufgabe 3 – Konsistenz nachweisen

1. Konsistenter Schnitt im laufenden Betrieb

Während die Überweisungen weiterlaufen, löst der Koordinator fortlaufend Snapshot-Runden aus. In jeder Runde wird die Summe aus allen gemeldeten Kontoständen und allen erfassten Kanalgehalten gebildet. Über alle Läufe hinweg ergibt diese Summe stets den Startbetrag S (z. B. 4000 bei $n = 4$), unabhängig davon, wie viele Überweisungen zum Schnittzeitpunkt unterwegs sind. Sobald eine Überweisung während des Schnitts unterwegs ist, wird ihr Betrag als Kanalzustand aufgezeichnet und fehlt dadurch nicht in der Gesamtsumme. Damit ist experimentell gezeigt, dass der Algorithmus einen konsistenten globalen Zustand liefert.

2. Naiver Schnappschuss und Bezug zum konsistenten Schnitt

Zum Vergleich wurde ein naiver Schnappschuss implementiert, der nur die Kontostände abfragt und die Kanäle ignoriert. Ist zum Schnittzeitpunkt eine Überweisung unterwegs, so hat der Sen-

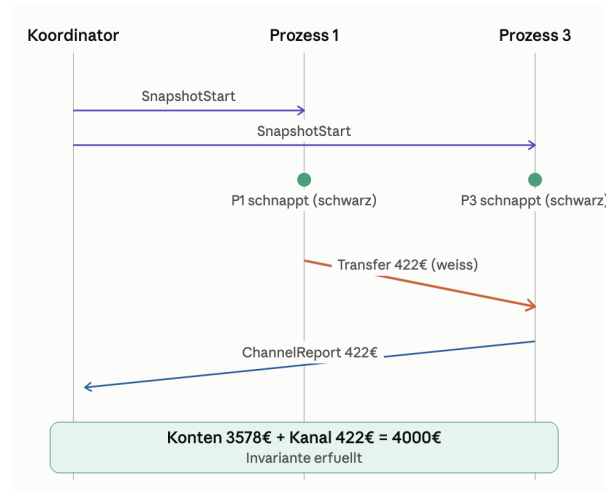


Abbildung 2: Invariante erfüllt

der den Betrag bereits abgezogen, der Empfänger ihn aber noch nicht gutgeschrieben. Der Betrag ist dann nirgends erfasst und die Summe unterschreitet S (im Versuch etwa 4515 statt 5000, also 485 “verschwunden”). Formal verletzt der naive Schnitt die Bedingung des konsistenten Schnitts $e_2 \in S \wedge e_1 <_k e_2 \Rightarrow e_1 \in S$: Das Empfangsereignis liegt hinter dem Schnitt, das kausal frühere Sendeereignis (die Kontoreduktion beim Sender) jedoch davor. Der Schnitt zerschneidet damit eine kausale Kette; das fehlende Geld ist die “Nachricht aus der Zukunft”. Der korrekte Algorithmus vermeidet dies, indem er genau diese unterwegs befindliche Nachricht dem Kanalzustand zurechnet.

3. Statistik und Nachteile

Für verschiedene n (2, 4, 8, 16) wurde die Anzahl der Kontrollnachrichten pro Snapshot-Runde gemessen. Sie setzt sich zusammen aus n **SnapshotStart**- und n **SnapshotReport**-Nachrichten sowie je einer **ChannelReport**-Nachricht pro unterwegs befindlicher Überweisung. Der Grundaufwand wächst also linear mit $\approx 2n$, zuzüglich der variablen Kanalmeldungen. Dies bestätigt die in der Vorlesung genannten Nachteile von Schnappschuss-Algorithmen: Das **Nachrichten-aufkommen** steigt mit der Prozesszahl, und der Ansatz ist “**nicht verteilt genug**”, da ein zentraler Koordinator alle Teilzustände einsammeln und auswerten muss – ein Single Point of Failure und ein Engpass bei großem n . Der Nachweis der Invariante gelingt trotz abgeschalteter FIFO-Zustellung, weil jede Basisnachricht ihre Farbe mitträgt.

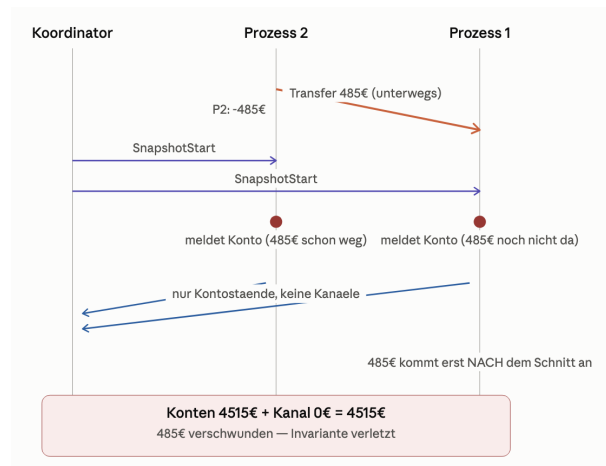


Abbildung 3: Invariante nicht erfüllt

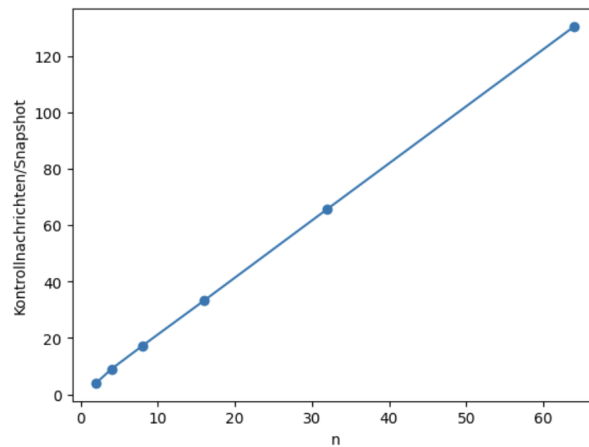


Abbildung 4: Anzahl Kontrollnachrichten verhält sich linear zur Anzahl der Prozesse

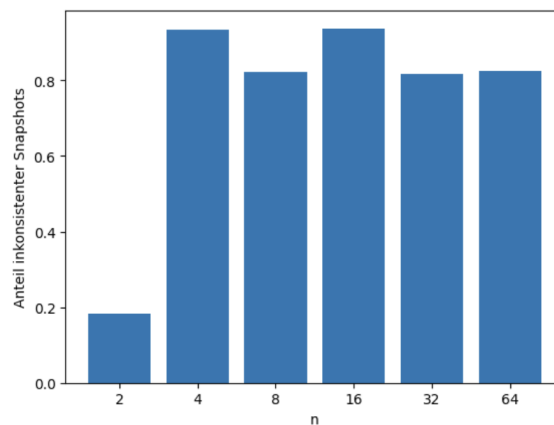


Abbildung 5: Die Anzahl inkonsistenter Snaps mit wachsendem n ist einigermaßen stabil.